FACULTY OF INFORMATION TECHNOLOGY

FIT1047 Introduction to computer systems, networks and security - S1 2026

Assignment 2 – MARIE Programming

Purpose	This assignment is mainly about programming in the MARIE assembly language. This will allow students to demonstrate their comprehension of the fundamental ways a processor works. In addition, basic knowledge about OS, processes and memory is assessed in an in-class demonstration. The assignment relates to Unit Learning Outcomes 2, 3 and 4.
Your tasks	**Part 1: Submit your reflections (Weeks 4 - 6).** See details below. **Part 2:** Disassemble and add comments to a MARIE program. **Part 3:** Write a MARIE program that can perform simple arithmetic calculations and display the result to the output and bitmap panels. **Part 4:** In-class in-person demonstration (Week 8 applied session [Australian Cohort]. Malaysian Cohort TBC).
Value	30% of your total marks for the unit. ● Part 2: 6 marks ● Part 3: 18 marks ● Part 4: 6 marks
Word Limit	See individual instructions
Due Date	**Part 1 - 3: 11:55 PM Friday 17 April 2026 (The submission links will be opened 72 hours before the deadline.)** **Part 4: Demonstration conducted during your official allocated Applied Session in Week 8 (27 April - 1 May) for Australian Cohort. Malaysian Cohort TBC.**

Submission	● Via Moodle Assignment Submission. ● **DRAFT** upload confirmation email from Turnitin is not a submission. You must click the submit button to accept terms and conditions in Moodle. Note that **DRAFT submissions will not be assessed**. ● Once the submission is confirmed, **any requests to revert it back to DRAFT** for resubmission will not be accepted. Also, **any incorrect, corrupted, empty or wrong file type** submission will not be assessed. **Please check carefully before confirming your submission.** ● Turnitin and MOSS will be used for similarity checking of all submissions. **Ignore Turnitin warnings or error messages for .mas files or other non-document files.** ● This is an individual assignment (group work is not permitted). ● In this assessment, you must **not** use generative artificial intelligence (AI) to generate any materials, modify or improve the content in relation to the assessment tasks for ALL parts (including Parts 1 - 4). ● You will need to **explain and extend your code** in the demonstration. (Part 4)
Assessment Criteria	Part 1 is assessed based on relevance of your reflections. Part 2 is assessed based on correctness of the code and the labels/comments. Part 3 is assessed based on correctness of the code, as well as the documentation/comments. Part 4 is assessed based on the understanding of the code you have written, and basic knowledge about operating systems, processes, and memory. See instructions for details. The marking rubric in Moodle shows an individual marks breakdown. If no or insufficient or irrelevant or improper reflections are submitted (Part 1), the overall mark for this assignment is capped at 15 out of 30 (i.e., 50% of the overall mark).
Late Penalties	● 5% deduction per calendar day or part thereof for up to one week ● Submissions more than 7 calendar days after the due date will receive a mark of zero (0) and no assessment feedback will be provided. ● Late penalty is automatically calculated based on Moodle submission. ***Therefore please note that even a late-submission for one second will incur a 1-day late penalty. We highly recommend you to submit at least 15 minutes before the submission deadline.***
Support Resources	See Moodle Assessment page
Feedback	Feedback will be provided on student work via: ● general cohort performance ● specific student feedback ten working days post submission

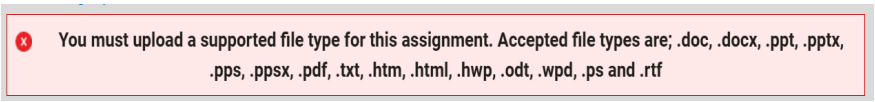
INSTRUCTIONS

This assignment has four parts. Make sure you read the instructions carefully.

For Part 1, collect your **reflections** for Weeks 4 - 6 from each week's Ed Lesson and create a single PDF document. **Submit your PDF through the Moodle Assignment 2 Part 1 activity. Please refer to the updated mandatory template.**

For Parts 2 and 3, submit your files through the **Moodle Assignment 2 Part 2 and Part 3 activities**. Each part requires the submission of **one or two .mas files inside a .zip file**.

You may see the following error message from Turnitin. You can ignore this error message!



An auto email message will be sent to you by Turnitin after you have uploaded the file(s). **Note that this email is NOT a confirmation email of your submission.** You must click the **submit** button in Moodle to accept terms and conditions and have your file(s) submitted successfully.

Part 4 is an in-class demonstration during your allocated Applied Class in Week 8 (Australian Cohort). Malaysian Cohort TBC.. Instructions will be available in Moodle and communicated via an announcement post in Ed.

Part 1: Reflection (Not marked, but cap on overall mark applies)

Collect your reflections for Weeks 4 - 6 from each week's Ed Lesson and create a single PDF document. You can simply copy/paste your reflections, but please add headings for each week. Please follow the ***updated mandatory** template (click here) as the structure of your reflections.

Different from Assignment 1, starting from Assignment 2 onwards, it is *mandatory* to use this updated template (click the above link to download) to write your reflection. The word count of reflection must be at least 200 words (excluding the headings provided in the template) in each week.

For each week, you must include and mention explicitly all three learning activities:

1) Ed lesson (can be the video or pdf notes)
2) Workshop
3) Applied.

Part 2: MARIE Disassembly (6 marks total)

Follow the link on Moodle to access your personalised MARIE memory screenshot for this task.

Important: Your memory screenshot is different from the one other students are working on. Only download the file (using the "Assignment 2 Part 2 Individual Disassembly Memory Map Download Link") while you are correctly logged into Moodle with your own student account. Submitting based on an incorrect Memory Map will be investigated as a potential breach of academic integrity.

Task 2.1: Disassemble the memory (3 marks)

Based on the memory contents, recreate the MARIE program that corresponds to your personalised memory screenshot. This is called "disassembling" the machine code, since it is the opposite operation of "assembling" the MARIE code into the binary memory contents.

For each memory cell, decode the instruction and (if applicable) the address that the memory cell is encoding. You can make the following assumptions:

- There is exactly one Halt *instruction* in the code
- Every memory location after the Halt *instruction* must be *data*
- Any memory location that contains the value 0 is *data* (even before the Halt *instruction*)

Below is an example of a memory screenshot and the corresponding decoded MARIE program:

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
000	5000	3005	6000	9000	7000	000A	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Disassembled program:

```
Input
Add 005
Output
Jump 000
Halt
DEC 10
```

Note: You need to decode the actual instructions. E.g., for the first memory location, HEX 5000 would not be a valid answer. The contents of all memory that follows the Halt instruction is considered to be data. Therefore, DEC 10 is the correct decoding of location 5 (instead of JnS 00A), and HEX 00A would also be correct. You do not need to list all the locations containing zeros starting from address 006 (these will be filled with zeros by the assembler anyway).

Hint: You can verify that your disassembled code is correct by entering it into the MARIE simulator, assembling it and comparing the memory contents to the screenshot you started from.

Task 2.2: Add labels (1 mark)

Now update the program you decoded in Task 2.1. Remove all hard-coded memory addresses by adding labels to replace all memory locations that are used as addresses in the program instructions. Labels should have meaningful names in the context of what the program does (i.e., not just A, B, C). For the example above, this could result in the following program:

```
MainLoop, Input
Add Ten
Output
Jump MainLoop
Halt
Ten, DEC 10
```

Hints: All data used by the code can be interpreted as integer numbers, so it's probably best to use the *Decimal* output mode. Run the code and observe the output. Ask yourself how the output relates to the given data. Use the step functionality in the MARIE simulator to understand step by step what the code does. Which parts are subroutines? Which parts are loops? What do the SkipCond instructions do? How are the different data values used?

Hints:
Some lines of code (instruction or data) are used as "placeholders" that do not contribute to the actual logic of the intended program. If such placeholders exist, they should still be retained in your disassembled code. In Task 2.3, you must include appropriate comments to indicate that the code is a placeholder and not used.

Task 2.3: Add comments (2 marks)

Comment the code based on your understanding of what the program/code segment does (not the definition of individual instruction). Comments should describe the overall function of the program, as well as the functions of different parts. E.g., if you identify a subroutine in the code, add a comment at the start of the subroutine that describes what it does, and whether it takes any arguments.

For this part (Part 2), submit a ZIP file containing a single .mas file containing your final code. If you attempted all three tasks, your .mas file should *not* contain three copies of the code - only submit the Task 2.3 code in that case (or only Task 2.2 if you did not attempt Task 2.3).

The Moodle submission activity for Part 2 contains a "quick check" function that you can use to check if your submission has the right structure. For Mac users, beware that there will be a hidden file when using the default "Compress" facility. The resulting zip file cannot pass the checker. Use the terminal approach to compress the file instead:

1. Go to "Terminal".
2. Go to the folder that contains the .mas file.
3. Type *zip <zip filename to be created> <your mas file>*. E.g., *zip P2.zip P2.mas*.

Part 3: MARIE Programming (18 marks total)

In this task you will develop a MARIE application that performs arithmetic calculation based on user input, display the result in the output panel, as well as the MARIE bitmap panel. We will break it down into steps for you.

Each task requires you to write **code, including appropriate labels** and **documentation**. On Moodle, you will find a **template** for Part 3. **Your submission MUST BE based on the given template only**, i.e., you must add implementations of your own subroutines into the template under relevant sections. The template already contains the **main program** that calls the subroutines.

Your code must contain readable comments and meaningful labels for your tutor / marker to understand the logic flow of your program (e.g., the **purpose** of a subroutine, `Jump` / `SkipCond` instruction etc.).

In-class demonstration (Part 4): You will be required to join a demonstration to show your code (and successfully run or execute your Part 3 tasks to be considered for grading) to your tutor during your applied session in Week 8 [Australian Cohort] (Malaysian Cohort TBC for the time). **Failure to demonstrate (**successfully run or execute your Part 3 tasks**) will lead to zero marks being awarded for the entire Part 3, regardless of your submission in Moodle**. In addition, during the demonstration (Part 4), you will also need to answer further questions about your submitted Part 3 tasks (see below for details) and MARIE and OS fundamentals.

Code similarity: We use tools such as MOSS and Turnitin to check for collaboration and copying between students. If you copy parts of your code from other students, or you let them copy parts of your code, this will result in a report to the Academic Integrity team. As a result, **you may receive a penalty such as 0 marks for the entire assignment, 0 marks for the whole unit, or in severe cases (such as contract cheating), suspension or expulsion from Monash University**.

Tasks Background

Arithmetic calculation

Arithmetic calculation is a fundamental computing task in which a program interprets numerical input, applies a selected operator, and produces a correct result. In this assignment, you will build a MARIE application that reads a user-entered arithmetic expression (in the "Inputs" panel as Unicode (UTF-16BE)) in a predefined format, that includes two positive integers separated by one operator from +, -, * or /, followed by "=?" (without any space in between). For example, the following inputs (as Unicode (UTF-16BE)):

```
123+456=?
159-23=?
59-83=?
```

```
12*46=?
132/11=?
138/13=?
```

Your program must extract the two integers (called the "Operands') and the operator from the input string, perform the required calculation, and output the result in a meaningful way. For addition, subtraction and multiplication, the output is a single integer. For division, the output consists of two integers: the quotient and the remainder.

Using the examples above, the following should be displayed on MARIE output panel (decimal) respectively,

```
579

136

-24

552

12
0

10
8
```

Tasks Descriptions of Basic Functions (Arithmetic calculation):

Task 3.1: Operands and operator extraction (2 marks)

Implement a subroutine SubInputAndExtractOperandsAndOperator to read an input string from "Inputs" and extract the two operands and the operator, and store them into three labels. For example, if the input is "*123+456=?*", *123*, *456* and *+* will be extracted and stored.

One possible implementation (you are free to use your desired approach as long as it fulfills the specification of this subroutine) is to traverse the input string from the beginning. It reads the first character, '1', and so stores the integer, 1, to the label of the first operand. It then reads the next character, '2'. Multiply the stored operand by 10 and add 2 to it (i.e., 12 = 1 * 10 + 2). You may use the multiplication subroutine you developed in your applied lesson. Do the same after reading '3'. Then, it encounters the operator, '+', in the string. So, we can conclude the first operand is 123. Store the operator into a label. For the second operand, read the characters that follow and perform the same operation using the same approach for the first operator, until '=' is encountered. Finally, we have *123*, *456* and *+* extracted and stored in three labels.

Hints: To convert a character digit to an integer, subtract a proper ASCII code value from the character being considered.

Input and output specifications of SubInputAndExtractOperandsAndOperator:

> **Input:** *none* (the string is obtained directly from the "Inputs" panel, in this subroutine)
> **Output:** operand1, operand2 and operator

For this task, you can assume the input string is always in a proper format, i.e.,

<operand1><operator><operand2>=?

Therefore, you do not have to perform any validation of the input string.

Task 3.2: Arithmetic calculation (3 marks)

Implement a subroutine SubCal to perform the arithmetic calculation based on the two operands and operator extracted in Task 3.1. The subroutine should read the values stored in the labels for operand1, operand2 and operator, determine which arithmetic operation is required, and store the calculated result into the appropriate label(s).

One possible implementation (again, you are free to use your desired approach as long as it fulfills the specification of this subroutine) is to compare the extracted operator with the ASCII codes of +, -, * and /, and then branch to the corresponding calculation logic/subroutine. For addition and subtraction, you may directly use the arithmetic instructions provided by MARIE. For multiplication, you can use your implemented subroutine (developed in your applied lesson). For division, implement a division subroutine using repeated subtraction. In the division routine, repeatedly subtract the second operand from the first operand until the remaining value would become negative. The number of successful subtractions is the quotient, and the remaining value is the remainder. For example, if we have 138/13, the subroutine should store 10 as the quotient and 8 as the remainder. Note that the remainder must be provided as a non-negative value.

Legends:

> + (Addition)
>
> - (Subtract)
>
> * (Multiplication)
>
> / (Division)

Input and output specifications of SubCal

> **Input:** operand1, operand2 and operator
> **Output:** result/quotient and remainder (for division)

For this task, you can assume that the extracted operands and operator are always valid, and that the operator is always one of +, -, * or /. Therefore, you do not have to perform validation in this subroutine.

Task 3.3: Text result output (1 mark)

Implement a subroutine `SubTextOutput` to display the arithmetic result produced in Task 3.2 in the MARIE "Output log" panel. The subroutine should read the calculated value(s) stored in the relevant label(s), and output them in a meaningful way.

For addition, subtraction and multiplication, the output of this subroutine is a single integer result. For division, the output consists of two integers: the quotient on the first line, and then the remainder on the second line.

Input and output specifications of `SubTextOutput`:

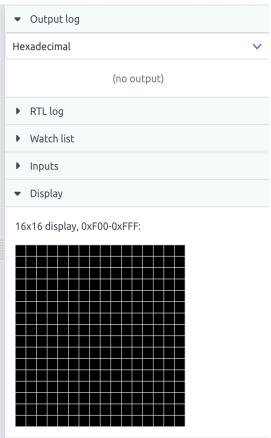
> **Input:** operator, result/quotient and remainder
> **Output:** *none* (value(s) shown in the "Output log")

Bit-mapped displays

Beyond producing the numeric answer in the "Output log", your program can also represent the result visually on the bitmap display, reinforcing the connection between internal data processing and external device output. So far, the only output capability we have seen in the MARIE system is using the `Output` instruction, which will print a single 16-bit value to "Output log". Many computers of course are capable of displaying arbitrary graphics, often in high resolution and great colour depth. In the lectures on input/output systems, we have seen that one way to implement this is to ***map*** a certain location of the memory to an output device, i.e., writing to that memory location (using, e.g., a `Store` instruction) causes the output to happen.

In the simplest form of graphics hardware, we can dedicate part of the RAM to be ***graphics memory***. Each memory cell corresponds to a ***pixel*** on screen, and the value in the memory cell encodes the ***colour*** of the pixel. That way, we can create arbitrary graphics by simply writing values into the memory.

The MARIE simulator has a feature called "Display", which you access from the list of tabs.

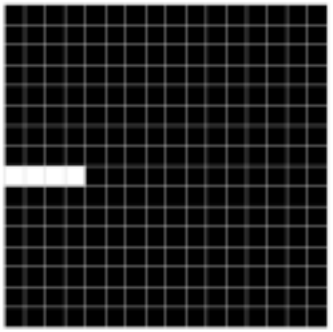


The "Display" shows the memory from address F00 to address FFF as a 16x16 pixel screen. The value in the memory locations represents the colour of the pixels. We will only use the colours black, represented as 0, and white, represented as FFFF. When you start the MARIE

simulator and assemble your code, the memory starting from location F00 is (usually) filled with zeroes, which means that the display is black. Let's now change the contents of the memory using some Store instructions:

```
Load White
Store 0F80
Store 0F81
Store 0F82
Store 0F83
Halt
White, HEX FFFF
```

After running this program, the display will look like this:



You can see that the first four pixels in the 9th row have now turned white.

Tasks Descriptions of Basic Functions (Bitmap Display):

Task 3.4: Clearing the display (1 mark)

Implement a subroutine SubClearDisplay that uses a loop to turn all pixels in the graphics memory white. Remember that the graphics memory ranges from address 0F00 to address 0FFF, and that white pixels are represented by the value FFFF.

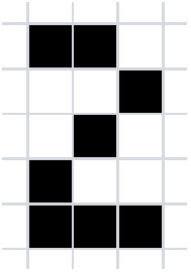
Input and output specifications of SubClearDisplay:

Input: *none*
Output: *none* (the display is cleared to white)

Task 3.5: Painting a character (2 marks)

The template for this task contains data for **bitmaps** of the character digits, '0' – '9', and the negative sign, '-', stored in the memory location starting from the label DigitFontStart. Each letter consists of 3x5 pixels of data. The first 3 words are the first row of pixels, the next 3 words are the second row, and so on. For example, the letter '2' is represented as:

```
0000 0000 FFFF
FFFF FFFF 0000
FFFF 0000 FFFF
0000 FFFF FFFF
0000 0000 0000
```



You can see the pattern here. The zeros "paint" the shape of the character '2'. We have highlighted the zeros in bold and red to make the pattern easier to see, but on the real display, the zeros would be black against the white background (FFFF).

You may first refer to Workshop 5 (Activity 3) to have revision!

Your task is to write a subroutine called SubPaintChar that paints a character into the graphics memory. The start of the subroutine needs to look like this:

```
PaintCharCharacter, HEX 0
PaintCharDisplay, HEX 0
SubPaintChar, HEX 0
```

In the PaintCharCharacter parameter, we pass the address of the first pixel data in the font for the character we want to paint. In the PaintCharDisplay argument, we pass the address of the top-left corner where we want to start painting in the graphics memory. For example, to paint the character, '1', starting from the second pixel in the second row, we could use the following code:

```
Load DigitFontAddr
Add NEXTDIGIT
Store PaintCharCharacter
Load PIXELOFROW2COL2
Store PaintCharDisplay
JnS SubPaintChar
Halt
PIXELOFROW2COL2, HEX 0F11
NEXTDIGIT, DEC 15
```

Note that the address 0F11 (label PIXELOFROWTWOCOLTWO) lies exactly 17 words after the start of the graphics memory. This means we are skipping the first row (16 words) and the first pixel in the second row (1 word). Note that PIXELOFROW2COL2 presets "the pixel of second column and second row" of the bitmap.

Here we use DigitFontAddr, which refers to the character, '0', and then add decimal 15 to it, to get the address of the character '1' (each character uses 3x5=15 words of memory, so '1' starts at DigitFontAddr + 15). For the other digit characters, we would have to add a

corresponding offset (NextDigit) into the font memory. Note that the pixels of '-' is stored after those of '9' in the template.

In order to paint a character in your subroutine, you may follow this "recipe":

- Your subroutine should contain two nested loops.
- Each character contains 15 pixels, so you need to loop through those 15 pixels, load each one from the font definition and store it into the graphics memory. This is the outer loop of your subroutine.
- After each set of 3 pixels (i.e., 3 columns in a row), you need to start in the next row of the graphics display. This means that if you were currently writing into graphics memory at address *X*, you now need to continue writing at address *X* plus the width of the display minus the width of a character. This is the inner loop of your subroutine.
- Once you have "copied" all 15 pixels from the digit font definition into the graphics memory, you can exit the subroutine.

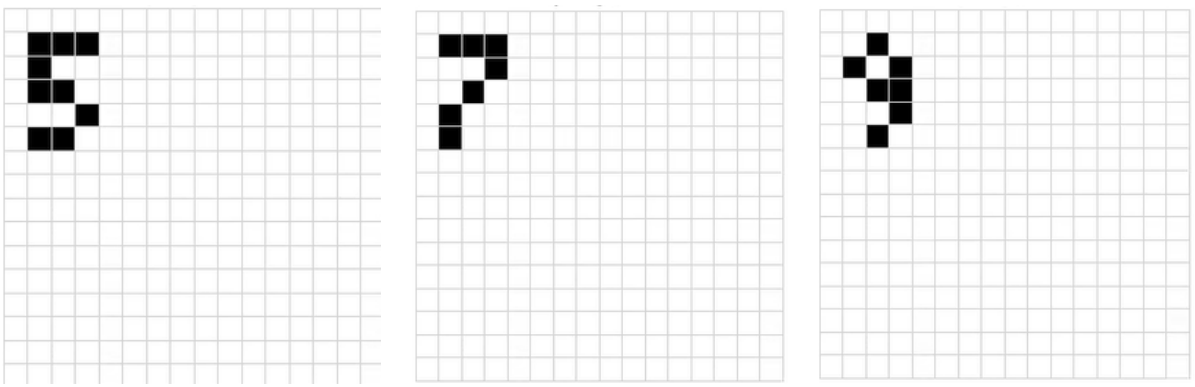
Input and output specifications of PaintCharCharacter:

Input: PaintCharCharacter and PaintCharDisplay
Output: *none* (a character is displayed in bitmap)

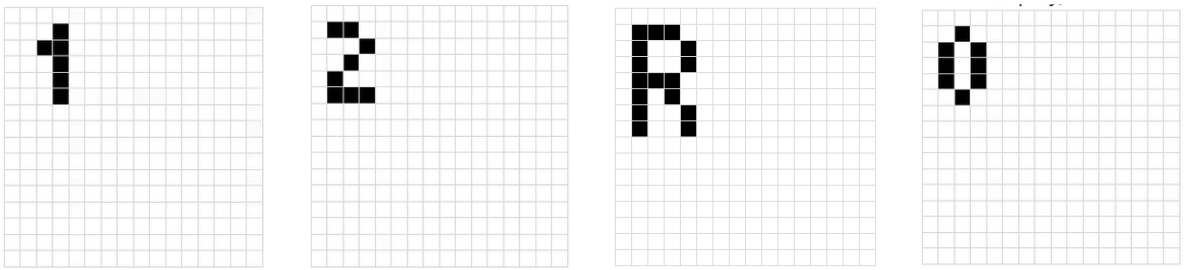
Task 3.6: Displaying the calculation result (2 marks)

In this task, implement a subroutine SubPrintCalResultToDisplay that uses the calculation result from Task 3.2, extract the digits of the result/quotient/remainder, clears the screen (using the subroutine of Task 3.4) and then writes the string to the display, one character, at a time. Each character of the string is painted using the subroutines developed in Task 3.5. After each character has been painted, clear the screen again and then paint the next character.

For example, for the output of the first example (i.e., 123+456=?), the result is 579, so we can see the following display sequence:



For the output of the fifth example (i.e., 132/11=?), the results are 12 and 0, so we can see the following display sequence:



*'R' means remainder.

Input and output specifications of SubPrintCalResultToDisplay:

Input: operator, result/quotient and remainder
Output: *none* (the calculation result is displayed in bitmap)

Hints:

1. To convert an integer to a character string, and suppose the integer is $y = 952$,
   i. Compute the number of digits, denoted by n, as follows: Divide y by 10 repeatedly until the quotient is 0.
      E.g.,
      952/10 -> 95,
      95/10 -> 9,
      9/10 -> 0.
      So, n is 3.
   ii. Extract the leftmost digit into an integer (1 - 9), followed by the second left digit (0 - 9) and so on. Here are the suggested steps:

      1. Set $i = n$, $k = 0$, create an array `Digit[]` (of size n) and set `Digit[0]=0, Digit[1]=0, Digit[2]=0,..., Digit[n-1]=0.`
      2. Set $i = i - 1$
      3. Divide y by 10^i *(10^i = 1 X 10 X 10 X 10 …. X 10. That is, 1 multiplied by 10 itself for i times.)* to get the quotient Q and the remainder R.
      4. Store Q into the array `Digit[k].`
      5. Set $y = R$ and set $k = k + 1$
      6. If y is not equal to 0, go back to Step 2.

      E.g. For 952,
      1. Set $i = 3$, $k = 0$, create an array `Digit[]` (of size 3) and set `Digit[0]=0, Digit[1]=0, Digit[2]=0.`
      2. Set $i = i - 1 = 3 - 1 = 2$
      3. Divide 952 by 10^2 ($10^2 = 1 \times 10 \times 10 = 100$). Quotient $Q = 9$ and remainder $R = 52$.
      4. Store *9* into `Digit[0].`
      5. Set $y = 52$ and set $k = k + 1 = 0 + 1 = 1$

6. Since *y* is not equal to *0*, go back to Step 2
    2. Set *i* = *i* - *1* = *2* - *1* = *1*
    3. Divide 52 by 10^1 (10^1 = *1 X 10* = *10*). Quotient Q = 5 and remainder *R* = *2*.
    4. Store 5 into `Digit[1]`.
    5. Set *y* = *2* and set *k* = *k* + *1* = *1* + *1* = *2*
    6. Since y is not equal to 0, go back to Step 2
        2. Set *i* = *i* - *1* = *1* - *1* = *0*
        3. Divide 2 by 10^0 (10^0 = *1*) [1 multiplied by 10 *zero* time ⇒ no multiplication is needed!]). Quotient *Q* = *2* and remainder *R* = *0*.
        4. Store 2 into `Digit[2]`.
        5. Set *y* = *0* and set *k* = *k* + *1* = *2* + *1* = *3*.
        6. Since y = *0*, exit the loop

The digits extracted have been stored in the array digit[]:

```
Digit[0] = 9
Digit[1] = 5
Digit[2] = 2
```

2. What to be displayed in bitmap depends on the operator. If it is addition, subtraction and multiplication, the *result* is to be displayed. If it is division, three parts will be displayed: *quotient*, 'R' and *remainder*. ***Please refer to the display example above.***

3. To paint a letter, you may implement a similar subroutine as that of Task 3.5. The letters are stored in the memory location starting from the label `FontStart`. Each letter consists of 4x8 pixels of data.

4. You will notice that it would be nice for each character to remain visible for a second or so before the next character is painted. There is no dedicated instruction for "waiting" purpose. Think about how to "make the CPU busy" before clearing the display for the next character to display.

Submit a ZIP file containing the .mas file with the code for Task 3.1 - 3.6, based on the template. Name the file as "P3T3_Basic.mas".

Tasks Descriptions of Advanced Functions:

Only work on this task when you have completed Tasks 3.1 - 3.6. **You will not be marked for Task 3.7 if the code you submit for Tasks 3.1 - 3.6 is incomplete or incorrect.**

Submit a separate (self-contained) .mas file, named as "P3T3_Advanced.mas", for this part, inside the same ZIP file that contains your submission for Tasks 3.1 - 3.6.

Task 3.7: Advanced functions (6 marks) **[Target for D/HD achiever]**

It will only be assessed if Tasks 3.1 - 3.6 are complete and correct! If you have not completed Task 3.1 - 3.6, no need to do this Task 3.7. To attain full marks, students should consider both technical difficulty and creativity.

For the final task, enhance the code you developed for Tasks 3.1 – 3.6 to make it having additional functions and more versatile and robust. The following enhancements have to be made in your code:

Task 3.7.1 (Negative integer handling). The input arithmetic expression should allow negative numbers. Each negative number must be enclosed by brackets. For example,

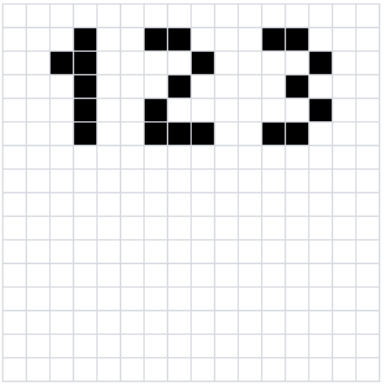
(-245)+(-24)=?
(-15)-(-74)=?
(-724)-82=?
(-45)*27=?
248*(-43)=?
(-90)*(-12)=?

Only the addition, subtraction and multiplication operations are required to take negative numbers into consideration. No need to implement division in task 3.7.

Task 3.7.2 (Error checking). Consider as many error conditions as you could, display the letter 'E' on both "Output log" and bitmap "Display" if it occurs. Errors can include, but not limited to:

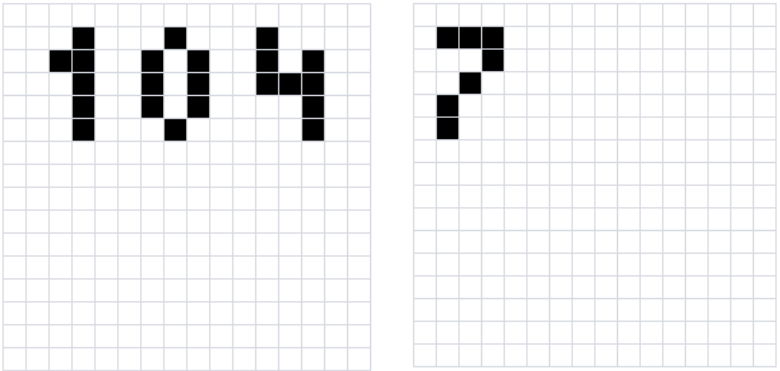
i. The input string contains any invalid characters, such as a space, $ and &.
ii. The input string contains two or more operators.
iii. The input string does not end with "=?".
iv. The first digit of an input is 0 (e.g., 012).
v. The denominator is 0.

Task 3.7.3 (Advanced bitmap display). If the number of digits of the result/quotient/remainder is equal to or less than 3, display all the digits in the bitmap at once. For example,



Display “1”, then followed by “2” (without erasing “1”) and “3” (without erasing “1” and “2”) on the same bitmap display.

If it exceeds three digits, display three digits at a time until all digits are displayed. For example, the following will be displayed if the number is 1047.



On top of 3.7.1, 3.7.2 and 3.7.3, use your creativity to implement additional error condition(s) not mentioned above, enhance the way of displaying the digits and other relevant functions to the three themes of 3.7. All the enhancements have to be technical sound as well. For example, ask yourself:

- Do the new features require complex constructs of looping and/or conditional branching, and/or complex memory management?
- While the implementation of the logic may be straightforward in high-level programming, how about in Marie?
- Are there any complex algorithms required for the new features? Why?

Add comments to the beginning of the program (1) to describe the extra feature(s) that you have implemented on top of 3.7.1, 3.7.2 and 3.7.3, and (2) to provide guidelines of how to run all the features for 3.7.

Overall Tasks 3.1 - 3.7 readability (1 mark)

In your program, provide sufficient comments to enable someone else (like the person marking your assignment) to understand the purpose of your code. For example, what are the purposes/functions of the entire program, the subroutines and the code segments?

Remarks:

- You may implement additional supporting subroutine(s) and create new label(s) of memory location(s) for Tasks 3.1 - 3.7.
- **0 mark** will be awarded if the program cannot be assembled. **Make sure your program can be assembled successfully**.
- A video with sample input/output cases for this part is provided for your reference on

Moodle.

Part 4: In-class demonstration (6 marks total)

You need to demonstrate your Moodle submitted code (successfully run or execute your Part 3 tasks) to your tutor in an in-class in-person demonstration session (**to be conducted in-person during your official allocated Applied session in Week 8 for Australian Cohort. Malaysian Cohort TBC later**). Failure to explain how your code works will result in 0 marks for the individual tasks that you cannot demonstrate **(**successfully run or execute your Part 3 tasks). More details about the demonstration will be announced later.

In addition, you will be asked questions about the following topics:

- Modify your MARIE code in certain ways to make it behave differently.
- Explain how the MARIE concepts (in which you were required to use for the individual tasks) work.
- Demonstrate how to access a list of processes and applications on your computer.
- Answer questions about operating systems, processes, and memory as covered in the pre-lesson materials, workshops and applied sessions.

These additional questions add up to 6 marks for this part.

Failure to attend the demonstration will result in <u>0 marks</u> for the entire Part 3 and Part 4, regardless of your submission in Moodle.

Submission Checklist: Please check before you submit.

Altogether, you should submit the following files:

Part 1:
One pdf file for Weeks 4 - 6 Reflections
Failure to submit this file will result in a maximum of 50% of marks in this assignment. **Remember to use the updated mandatory template to write your reflection. It is a *hurdle* requirement!**

Part 2:
One .zip file containing one P2.mas file with your final code for Tasks 2.1 - 2.3. Do not submit one .mas file *per each subtask*. Do not submit any pdf file in this part.

Part 3:
One .zip file containing one P3T3_Basic.mas file with your final code *based on the template*, containing the code for Part 3 Tasks 3.1 – 3.6. Do not submit one .mas file *per each subtask*. Do not submit any pdf file in this part.

Optionally, one additional P3T3_Advanced.mas file inside the same .zip file for Task 3.7.

Failure to submit the correct type of files (e.g., submit a word file instead of a pdf file in Part 1, or a pdf file in Part 2 or Part 3), or missing any file in your submission (e.g., your weekly reflection) will result in significant marks deduction.

Please check before you submit.
You CANNOT revert any submission.

Draft submissions are not assessed.